Appendix D — Enterprise agent system study guide (v2)

Source: `study_guide_enterprise_system_v2.md`

Enterprise AI Agent Platforms: A Staff-Level Study Guide

1. Core problem being solved

Enterprise software vendors increasingly let end users interact with business systems through natural-language "agents" instead of fixed UI flows or hand-written integrations. An agent takes a free-text request, decides what the user wants, retrieves relevant business data, chooses among a set of callable capabilities ("tools" or "actions"), and either answers or performs a side-effecting operation (create, update, delete) on the user's behalf.

This introduces a problem that conventional application architecture has never had to solve at this scale: **the component that decides what to do (a large language model) is non-deterministic, cannot be exhaustively tested, and is influenceable by anything in its input** — including data that was never meant to be an instruction. Historically, "what the system does" was authored by engineers and was a fixed, auditable code path. Now "what the system does" is *synthesized at runtime* from a prompt, a tool catalogue, and retrieved context, and the exact same input can legitimately produce different outputs on different runs.

The core engineering problem is therefore: **how do you build a system where a probabilistic, untrusted reasoning component can propose actions, while every actual effect on real data and real systems is governed by deterministic, testable, fail-closed controls that do not depend on the model behaving correctly?**

This generalizes a much older problem in distributed systems and security — the *confused deputy* problem (Norm Hardy, 1988): a component that holds more authority than the party invoking it can be tricked into misusing that authority. An LLM-driven agent is a new, particularly potent instance of this pattern, because the "confusion" can now be induced through natural language rather than a crafted API call.

2. High-level architecture

A production agent platform is best understood as a **pipeline of zones with different trust levels**, not a single service:

```
[ End User / Channel ]
        |
[ Agent Runtime: intent classification, planning, orchestration ]
        |
[ Policy / Trust Boundary Layer: input & output filtering, masking, injection defense ]
        |
[ Model Provider: the LLM itself ]
        |
[ Tool / Action Execution Gateway: schema validation, authorization, rate limiting ]
        |
[ Systems of Record: transactional data, search/retrieval indexes, external APIs ]
        |
[ Human-in-the-loop approval, where required ]
        |
[ Audit / Observability substrate, cutting across every layer ]
```

Key architectural properties:

- **Layered decomposition.** A useful mental model splits the platform into: (a) an **experience/channel layer** (web, chat, voice, API); (b) an **agent definition layer** (the configured topics, instructions, and available actions — mostly declarative, not code); (c) a **runtime/orchestration layer** (the planner loop that classifies intent, retrieves context, selects tools, and composes responses); and (d) a **tool/execution layer** (the code that actually touches data).
- **The reasoning plane is untrusted; the control plane is trusted.** The single most important structural decision in any such system is a hard boundary between the component that *decides* (the model) and the component that *authorizes and executes* (deterministic code). Everything crossing this boundary is treated as *data*, never as *authority*.
- **Grounding/retrieval is a first-class subsystem**, not an afterthought — a pipeline that ingests, harmonizes, indexes, and serves business data back into the model's context window, each stage with its own latency, staleness, and access-control characteristics.
- **A standardized tool-calling protocol** (of the kind popularized by open, vendor-neutral specifications for connecting language models to external tools) is increasingly used to decouple "tool providers" from "agent runtimes" — a host application talks to a client library, which negotiates capabilities with independent tool servers over JSON-RPC. This is valuable architecturally but introduces its own trust boundary at every server connection.

3. Component responsibilities

Generic role	Responsibility	Analogous to
Agent Runtime	Runs the plan → act → observe loop per conversational turn; classifies intent; decides which tool to call and with what arguments; composes the final response. Its output is always a *proposal*, never an authorized effect.	Orchestrator / controller in a workflow engine
Policy Engine (Guardrail / Trust Layer)	Platform-level checkpoint that every model interaction passes through: masks sensitive data before it reaches the model, screens for toxic/unsafe content, screens for injected instructions, and produces an audit record. Fails closed if unavailable.	API gateway policy layer / WAF for LLM traffic
Retrieval Service (Grounding Pipeline)	Ingests business data, chunks and embeds it, maintains a search index, and serves permission-filtered context back to the runtime so the model reasons over real facts rather than invented ones.	Search/RAG subsystem
Tool Gateway (Execution Gateway)	The single choke point where a proposed tool call becomes a real effect: validates the call's schema, resolves the entities referenced, re-derives authorization independent of the model, applies rate limits, ensures idempotency, routes to human approval if required, executes, and writes an audit record.	Command bus / API gateway with policy enforcement
Authorization Layer	Deterministically answers "does this identity have the right to do this, to this data, right now?" Independent of anything the model said. Composed of object-level, field-level, and record-level checks plus any domain-specific entitlement rules.	RBAC/ABAC policy decision point (PDP) + enforcement point (PEP)
Human-in-the-Loop (Approval) Service	Durable workflow that routes high-consequence proposals to a human approver, shows the approver the *actual* validated arguments (not a paraphrase), and blocks execution until a decision is recorded.	Approval/workflow engine
Idempotency / Concurrency Store	Tracks idempotency keys and version tokens so retries cannot double-apply an effect and concurrent writers cannot silently clobber each other.	Distributed lock / dedup table
Audit / Observability Substrate	Immutable, correlated record of every decision (not just every outcome): what was proposed, what policy decided and why, who approved, what actually happened.	Centralized structured logging + tracing backend

Generic role	Responsibility	Analogous to
Tool/Action Definitions	Typed contracts (name, description, parameter schema) that the model reads to decide what to call. Treated as a supply-chain artifact — reviewed, versioned, and pinned like any dependency.	Service interface / API contract

4. Request/data flow

A representative end-to-end trace for a request that both reads and mutates data (illustrative, not a specific product's exact behavior):

1. **Ingress and authentication.** The user's utterance arrives over some channel with an authenticated session/identity already established.
2. **Guardrail scan (input).** The Policy Engine screens the raw utterance for injection patterns before it becomes part of any prompt.
3. **Intent classification / planning (turn 1).** The Agent Runtime's planner selects a bounded task scope ("topic") and reasons about what capability is needed.
4. **Grounding retrieval.** If context is needed, the Retrieval Service runs a query **scoped to the requesting user's own access**, not the agent's, and returns a bounded set of permission-filtered chunks.
5. **Guardrail scan (context assembly).** Retrieved content is inserted into the prompt as clearly delimited, non-authoritative *data*.
6. **Tool selection (turn 2).** The model emits a structured proposal: a tool name plus arguments. This is a *request*, not an executed action.
7. **Schema and semantic validation.** The Tool Gateway rejects anything that doesn't conform to the tool's typed contract, then checks whether the proposal makes sense given current world state (e.g., the target record still exists, the subject is still active).
8. **Authorization (independent re-derivation).** The Authorization Layer checks — using the real caller's identity, never the model's claim — whether this specific action on this specific target is permitted right now.
9. **Policy decision.** A deterministic policy engine returns one of: permit, deny, or require-approval.
10. **Human approval (conditional).** If required, the proposal is queued; a human sees the actual arguments and either approves or rejects.
11. **Idempotent execution.** The gateway executes exactly once per idempotency key, under the caller's own authority wherever possible.
12. **Output guardrail scan.** The result is screened before being fed back into the model for response composition (masking, toxicity, leakage checks).
13. **Response composition (turn 3).** The model summarizes the outcome for the user.
14. **Audit write.** Every step above — not just the final outcome — is written to an immutable audit log, correlated by a single trace/turn identifier.

The critical invariant across this flow: **authorization is re-checked at execution time, against live state and the real caller's identity — never inferred from what was decided at planning time.**

4.1 A granular walkthrough — input, output, state, and attack surface per step

For a maximally detailed picture, the same request/response cycle above can be decomposed into roughly twenty finer-grained steps, each with its own input, output, state, and — where relevant — named attack surface. This granularity is the format that reads as "I've actually built this," because it carries a named attack surface per step rather than staying at the level of generic vocabulary; it's also useful whenever the question is really "where exactly could this go wrong" rather than "walk me through the architecture."

1. **User input** — arrives over a channel surface (chat, console, programmatic caller). The channel's own transport security applies; prompt injection can be embedded directly in the utterance; a channel outage is itself a failure mode worth designing for.

2. **Authentication** — input: credentials/token. Output: an authenticated principal. State: session/token, deterministic. Attacks: token theft, an over-privileged integration, confused deputy. Retry path: re-run the auth flow; don't silently degrade.
3. **Session** — the runtime's own session manager. Output: a session handle holding conversation state, deterministic. Attack: session fixation or token leakage.
4. **Runtime and configuration load** — the runtime loads the agent's own configuration for this caller. Input: agent ID. Output: an in-memory agent definition. **The configuration itself is a security boundary** — authorization effectively starts here, at what configuration the runtime is even allowed to load for this caller, before the first model call.
5. **Context assembly** — input: session + principal. Output: a context bundle.
6. **Topic/intent selection** — the reasoning engine classifies the request against a bounded task scope (inputs: utterance, context, scope definitions). Output: the selected scope.
7. **Instructions** — the scope's own instructions merged with context. Output: the augmented reasoning prompt.
8. **Grounding** — determining what the plan needs. Output: grounded, retrieved data merged into context. **Security boundary:** what gets fetched is itself plan-influenced, which is exactly why it has to be filtered before retrieval, not after.
9. **Retrieval** — the step that actually determines the correctness of everything downstream; a bad retrieval here corrupts every later stage even if every later stage behaves perfectly.
10. **Tool/action discovery** — a scoped, topic-relevant action catalog, not the full catalog (the same shortlist/funnel principle that keeps LLM cost and latency decoupled from catalog size, generalized to any request).
11. **Action selection** — the planner selects an action plus its arguments.
12. **Parameter generation** — the model emits typed arguments — concretely, something like a set of record IDs — never free text.
13. **Schema validation** — reject malformed input cheaply, before it's trusted for anything downstream.
14. **Authorization** — grounded in the platform's own action/permission model, never the model's say-so. Attack: confused deputy — this is the step it's actually aimed at.
15. **Confirmation** — a deterministic gate in front of execution, not a model decision.
16. **Execution.**
17. **Downstream API/data call** — the actual persistence layer.
18. **Result & observation** — the runtime captures what was actually executed, not what was merely planned.
19. **Re-planning** — if the result changes what's needed next.
20. **De-masking for display** — anything masked going into the model gets selectively unmasked for the human-facing response, under its own policy.
21. **Audit logs** — and everything upstream of this, not just the final outcome.

What the trust/policy layer is specifically doing across these steps reduces to three concrete jobs, worth having as a self-contained answer: (a) masking of PII before the prompt ever reaches the model, not after; (b) zero data retention at any third-party model provider the prompt passes through; (c) prompt defense and an audit trail tied to a concrete enforcement point in the pipeline, not floated as an abstract policy.

A shorter, compressed sketch of the same journey, if something faster to draw is useful: `Authenticate → Context → ClassifyIntent → Grounding → RiskClassify → Plan ⇄ ValidatePlan (replan loop) → Execute → Observe`, with a second branch off Plan for `Needs more info → Replan → ProposeWrite → Authorize → Execute → Verify → Audit → Respond`. The one stage this compressed version names that the two longer versions leave implicit is **RiskClassify**, sitting right after Grounding and before Plan — worth drawing as its own explicit box, since it's exactly where risk tiering (5.8) plugs into the pipeline, rather than being folded silently into the approval stage alone.

4.2 The four authority boundaries — where authority actually changes hands

A tighter way to answer "where exactly can this go wrong" than walking the full step-by-step list above: name the four handoffs where authority crosses a boundary, because each one is a place a confused-deputy or over-

authorization bug can hide.

```
Human → (resolves running user) → Agent Runtime → (prompt + tool schema) → Planner → Tool Args → Tool → Data Store
```

1. **User → Agent** — the running user is decided once, at the start of the interaction, and is rarely re-decided for the rest of it. For any long-lived session, ask explicitly whether re-authentication or re-derivation ever happens, or whether the original decision is trusted indefinitely.
2. **Planner → Router** — there is typically no re-authentication at this hop by default; the router trusts that the planner's caller context is still the one established at step 1.
3. **Router → Tool** — the hop that actually converts *end-user identity* into *system authority*. This is exactly where the confused-deputy problem's strongest mitigation (execute under the caller's own authority, 5.3) has to hold.
4. **Tool → Data Store** — the hop most likely to be assumed safe and actually isn't. Object-level and field-level security are not automatically enforced just because the tool layer checked something upstream — the data-store call itself has to independently re-check, or this is exactly where a "we validated it in the tool" claim quietly stops being true.

Given any specific bug report or hypothetical, naming which of these four boundaries actually failed is a much sharper diagnostic than "authorization broke somewhere."

5. Security model

5.1 Threat framing

The system is best modeled with a standard property-violation taxonomy (spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege) applied at every point where data crosses a trust boundary: user→channel, channel→runtime, runtime→model provider, runtime→tool gateway, gateway→data store, gateway→external tool/server, runtime→human approver, and retrieval-store→runtime. Each crossing has its own realistic attack and its own primary control; the recurring finding is that **the highest-leverage controls are (1) always executing tool calls under the real caller's authorization context, (2) treating every inbound block of text — user input, retrieved documents, and tool output alike — as untrusted, and (3) failing closed whenever a security-relevant dependency is unavailable.**

This maps cleanly onto two public, vendor-neutral bodies of work worth knowing by name in any system-design discussion: the OWASP Top 10 for LLM Applications (categories such as Prompt Injection, Excessive Agency, Sensitive Information Disclosure, Supply Chain, Unbounded Consumption) and MITRE's adversarial-technique knowledge base for AI systems (an ATT\&CK-style catalog of concrete attacker techniques against ML/LLM systems). Both are useful as checklists and shared vocabulary; neither tells you *where* in your specific architecture to put a control or whether that control is deterministic — that part is still an architecture job.

5.2 Authentication vs. authorization

These are different questions and must never be conflated. **Authentication** proves who is making a request. **Authorization** decides whether that identity is entitled to a specific effect on specific data, right now. A system that treats "the user is logged in" as sufficient to let an agent act broadly on their behalf has confused the two — this is exactly the gap that produces confused-deputy vulnerabilities.

5.3 The confused deputy problem

Coined by Norm Hardy in 1988: a program that has more authority than the party who invoked it can be manipulated into misusing that authority on the invoker's behalf. An AI agent is a near-perfect instance of this: it often runs under a service identity or elevated integration credential with broad reach across the business's data,

while individual end users have much narrower rights. If the agent doesn't re-check the *end user's* authority before acting, the user can reach data or effects through the agent that they could never reach directly — and a prompt injection makes this exploitable by a third party, not just a curious end user.

Mitigations, in order of strength:

1. **Execute under the caller's own authority** wherever the caller has a downstream identity — the single strongest fix, because it makes the agent's blast radius mathematically no larger than what the human could already do.
2. Where elevation is unavoidable (e.g., a background lookup against an HR system the user has no direct access to), use a narrowly-scoped, separately-audited service credential rather than a broad standing one — and re-verify the user's authority for anything sensitive derived from that lookup.
3. Never let the model's own assertions ("the user is an admin," "policy allows this") substitute for a real permission check.
4. Bind any delegated token to the specific resource/audience it's meant for, and never forward a token upstream to another service unmodified — otherwise that service inherits whatever the original token could reach.

A concrete worked example is worth walking through by name rather than staying at the abstract definition: a user — call her Dana — sends a request through the agent runtime and execution layer (the "deputy"), which itself holds special authority (an integration or service-account permission broader than Dana's own) reaching a protected resource Dana shouldn't reach on her own. Dana's only lever into this chain is the prompt itself. If the deputy ever uses its *own* authority to satisfy that request without re-checking whether Dana specifically is entitled to the result, that's the vulnerability, full stop — it doesn't require a clever exploit, just a system that forgot to recouple authority to the actual requester. The defense is exactly that recoupling: every hop in `Dana → Agent → Tool` re-derives an authorization check against Dana, never against the agent's own standing credential, before the data source is ever touched.

5.4 Prompt injection

Direct injection: the user's own input tries to override system instructions ("ignore your rules and…"). **Indirect (stored) injection**: instructions are smuggled in through content the agent *reads* rather than through the conversation itself — a poisoned document in a retrieval index, a malicious field value in a record, an adversarial tool description, or text embedded in an external tool's response. Indirect injection is more dangerous operationally because the end user may be entirely innocent; the attacker's payload rides in through "trusted" data paths.

Defense is layered, not a single fix: input scanning, strict separation of "instructions" from "data" in how the prompt is assembled, treating all retrieved/tool-originated text as non-authoritative, constrained/typed tool schemas that limit what a manipulated plan can actually do, deterministic authorization that doesn't care what the model "decided," and egress controls on where output can go. **No layer alone is sufficient, and no combination of layers claims to make injection provably impossible** — the goal is to raise the cost of a successful attack and bound the damage of one that partially succeeds, not to guarantee immunity. Overclaiming "we've solved prompt injection" is itself a tell worth avoiding.

5.5 Hard guardrails vs. soft guardrails

Soft guardrails are behavioral: instructions in a system prompt telling the model what not to do. The model *may* comply, and can be talked out of compliance through injection, jailbreak phrasing, or simple misgeneralization. **Hard guardrails** are deterministic: code-level checks the model cannot argue with — schema validation, authorization checks, allow-lists, rate limits, human-approval gates. **Every security-relevant guarantee must rest on a hard guardrail.** Soft guardrails are useful for quality and nuisance reduction, never for anything where a bypass has real consequences. The test to apply to any claimed control: "can the model talk its way past this?" If yes, it is soft, and cannot be the last line of defense.

5.6 Secure tool/action design

A tool definition is not documentation — it is the interface the model uses to decide what to call and how, and it is therefore part of the security boundary. Guidelines:

- **Prefer narrow, verb-specific actions** ("revoke this specific grant") over generic, broadly-parameterized ones ("update any field on any record"). A narrow tool is easy to authorize, easy to audit, and has a provable blast radius; a generic tool hands the model a capability whose actual reach is bounded only by whatever policy code you can write to constrain it — and any gap in that policy is directly exploitable.
- **Validate structure, then meaning, then effect.** Reject malformed input cheaply (schema); then check that the request makes sense given current state (semantics); then authorize; then execute. Each layer narrows what the next layer has to trust.
- Treat every optional parameter, relaxed "required" flag, or broadened enum as an **API-contract and security-boundary change**, because it can silently widen the set of records or effects reachable through a tool — even when nothing in the backing authorization configuration changed. Any tool-schema change should trigger a full re-run of security-relevant regression tests, not just a functional smoke test.
- **Never expose a raw/low-level query or update capability** to the model. Every exposed capability should be a bounded, purpose-built operation.

5.7 The execution gateway pattern

A single, non-bypassable choke point through which every proposed tool call must pass before it has any real-world effect. A representative stage sequence: schema validation → semantic/entity-resolution validation → authorization (re-derived from the real caller) → rate limiting → idempotency-key check → human-approval routing (if required) → execution under the caller's authority → output normalization → audit write. The value of this pattern is that it centralizes every security-relevant check in one reviewable place, rather than trusting each individual tool's implementation to get all of them right independently.

A tighter mnemonic — the gateway as eight sequential gates. If asked to enumerate the gateway's checks by name rather than describe it as one blob, this ordering is worth having memorized exactly:

1. **Tool allowlist** — is this tool even allowed for this user?
2. **Argument schema validation.**
3. **Decide whose permissions matter** — always the real caller's, never the model's claim about who's asking.
4. **Execute the authorization decision** — deterministic, code-level.
5. **Mint a capability** — a short-lived, narrowly-scoped token for this one operation, not a standing credential.
6. **Egress control** — confirm secure/sensitive fields aren't about to leave the boundary unmasked.
7. **Tamper-evident audit** — record what happened, immutably, before or alongside the effect itself.
8. **Confirmation on consequential writes** — the human-approval gate (5.8), for the tier of action that needs it.

The layered-bounding line worth having verbatim, since it explains why no single gate has to be perfect on its own: "*The topic bounds which actions are even visible; the schema bounds what each action can express; and the backing code, running as the authenticated user, bounds what can actually be executed. Three independent, narrowing constraints — not one gate that has to catch everything.*"

The same eight gates can also be organized as a shorter, seven-item punch list rather than a sequential pipeline, useful when the question is asked as a checklist rather than a flow: authentication → authorization (object/field/record-level policy) → schema validation (oversized or malformed arguments rejected) → API/tool allowlisting → transaction constraints (data-store integrity, atomicity, referential rules) → confirmation gate → network restrictions. Use whichever framing matches how the question was actually asked.

5.8 Human-in-the-loop (HITL) and risk tiering

Autonomy should decrease as consequence increases. A workable tiering:

Tier	Example	Typical gate
T0 — Read-only	Look up a record	Fully automatic
T1 — Reversible, low-impact write	Update a status field	Automatic, logged
T2 — Consequential write	Issue a moderate refund	Confirmation required
T3 — High-value / hard-to-reverse	Grant a sensitive permission	Human approval, distinct approver from requester
T4 — Bulk / irreversible	Mass deletion, bulk permission change	Approval + rate limiting + hard caps

Two failure modes to design against explicitly: **approval fatigue** (if everything requires approval, humans rubber-stamp, and the "human gate" becomes decorative — solve with risk-based routing so only genuinely high-risk proposals reach a human, and show the approver a real diff, not a paraphrase) and treating human approval as *replacing* authorization rather than *complementing* it (a human clicking "approve" on an unauthorized action does not make it authorized — the human is approving a proposal, and the proposal itself still has to have passed independent authorization before it was ever shown; a system that skips re-checking permissions after approval has treated approval as a substitute, not a complement).

5.9 The grounding threat model — eight named attack vectors

Retrieval is not just a quality problem — it is its own trust boundary with its own attack surface, distinct from prompt injection generally (5.4) and from tool-level attacks (6.2). Eight vectors worth being able to name and mitigate on demand:

1. **Retrieval poisoning** — an attacker seeds a document crafted to rank highly for specific target queries, so it gets pulled into context even though it isn't actually authoritative. Mitigation: restrict *who* can create indexable content in the first place — a content-governance control, not a retrieval-ranking fix.
2. **Prompt injection via retrieved data** — the indirect-injection case from 5.4, specifically through the grounding path. Mitigation: prompt-defense policies applied to retrieved content as a class, not just to user input.
3. **Stale grounding** — the index hasn't been refreshed since the source changed. Mitigation: read-after-write (or a live query) for anything grounding-sensitive, rather than trusting a possibly-stale index.
4. **Cross-user leakage** — retrieval isn't scoped to the requesting user at all.
5. **Cross-tenant leakage** — the index or vector store is shared across tenants without real isolation. Mitigation: tenant isolation, ideally with tenant-scoped credentials or keys — the same principle as executing under the caller's own scoped authority (5.3), restated for the index itself rather than just the query path.
6. **Unauthorized retrieval** — a caller's own retriever implementation bypasses the platform's sharing/permission rules. Mitigation: enforce the running user's access inside the retrieval call itself, never assume an upstream check already covered it.
7. **Metadata leaks** — a chunk includes prepended or internal fields that were never meant to reach the model or the user.
8. **Malicious documents** — an uploaded file carries hidden text, homoglyphs, or embedded instructions designed to be extracted during parsing. Mitigation: parser hardening — treat document parsing itself as a place attacker-controlled content can smuggle a payload through, not just the extracted text.

The line that ties all eight together, worth stating verbatim: "*Every grounding result has to pass through the platform's own sharing, field-level-security, and access-control model before it reaches the prompt — retrieval doesn't get a side door around the permission system just because it's a vector search instead of a direct query.*"

Two distinctions worth having crisp, because they're easy to blur under pressure:

- **Masking and authorization solve different problems.** Authorization controls *which data you may access at all*; masking controls *which sensitive fields are exposed even after legitimate access*. A field can be authorized and still masked (you can see the record, not a sensitive field on it); the two checks are independent and both have to pass.

- **Grounding improves knowledge quality; it does not improve — or substitute for — authorization.** This is exactly the mistake vector #6 above describes: assuming that because retrieval *found* something relevant, the caller was *entitled* to it.

6. Agent/tool interaction model

- **The model proposes; deterministic systems dispose.** The model's output when it "calls a tool" is a structured request — a name and arguments — not an executed effect. The runtime, not the model, decides whether and how to execute it.
- **Tool descriptions are untrusted supply-chain input.** The text a tool exposes to the model to describe when/how to call it is itself read and acted on by the model, so a compromised or malicious description is a viable attack vector ("tool poisoning"). Descriptions should be pinned, hashed, and reviewed like any dependency, with drift after approval treated as suspicious ("rug pull").
- **Tool outputs are untrusted input on the way back in.** Anything a tool returns re-enters the model's context and can carry an injected instruction, exactly like a malicious retrieved document. Output must be validated/sanitized before being trusted for further reasoning or shown to the user.
- **A standardized tool-calling protocol** (of the kind now common for connecting agents to external tool servers) typically exposes three primitive kinds of surface: callable **tools** (functions with side effects or lookups), readable **resources** (context data), and reusable **prompt templates**. It runs over a base remote-procedure-call protocol and includes a capability-negotiation handshake, plus an authorization model built on delegated, audience-bound tokens (so a token minted for one server cannot be replayed against another) and an explicit prohibition on blindly forwarding a client's token to an upstream service.
- **Connecting to any external tool server extends your trust boundary to that server's code and everything upstream of it.** Trust is not transitive and not static — a server can change a tool's behavior after it was reviewed and approved. Practical controls: an explicit allow-list of connectable servers, least-scoped and audience-bound credentials per connection, treating both descriptions and outputs as untrusted, and periodic re-review for definition drift.
- **Fine-grained tools vs. broad/generic tools is the single most consequential interface design decision.** A generic "update any field on any record" tool is convenient to build but hands the model a privilege-escalation primitive: the model effectively parameterizes what data gets touched. The mature middle ground — when a generic tool is unavoidable — is a **capability-scoped generic tool**: constrained by an explicit allow-list of the specific objects/fields it may touch in a given context, with hard value constraints enforced as policy rather than as prompt instructions, and platform-level authorization still acting as the non-negotiable backstop underneath the model's choice.
- **Multi-agent (supervisor/specialist) architectures** trade a larger coordination and trust surface (every inter-agent message is a new untrusted channel, and a specialist's hallucination can propagate unchecked if the supervisor over-trusts it) for separation of concerns and parallelism. Prefer a single, well-scoped agent unless the domains are genuinely distinct; if multi-agent is used, treat every inter-agent handoff exactly like an external, untrusted input.

6.1 A standardized tool protocol's own architecture — three parts

Worth being able to draw from memory, since a "connect the agent to an external tool" question often wants this specific shape rather than the generic bullet above:

- **Host** — acts as the container and coordinator. Creates and manages multiple client instances, enforces security policy and consent requirements, and holds the full conversation history — the host is where cross-server context actually lives, not any individual client or server.
- **Client** — each client maintains an isolated connection to exactly one server and one stateful session per server; the client is where protocol negotiation and capability exchange happen.
- **Server** — provides specialized context and capabilities by exposing three primitives: **resources** (readable, declared by the server), **tools** (actions/functions the model can invoke), and **prompts** (reusable templates the server declares; the client can list them).

Transport is typically JSON-RPC 2.0 — every message carries an id, a method, and params. The initialization handshake, in order: the client sends its protocol version, capabilities, and client info; the server responds with its own protocol version, its capabilities, and its identity; the client confirms with a notification. Two things worth stating if asked why this matters: version negotiation happens once but its result is carried on every subsequent request, and capability negotiation is the actual heart of initialization — it determines which primitives either side is even allowed to invoke for the rest of the session, not a formality before the "real" protocol starts.

A less commonly discussed half of the protocol is client-side primitives: **roots**, **sampling**, and **elicitation** — the server can ask the client's own model for help generating something (sampling), and can ask the human for more information mid-tool-call (elicitation). Concretely, the server "asks the host's brain for help" rather than assuming it has to solve everything itself — a real inversion worth naming if the question is whether the protocol is bidirectional rather than just "client calls server's tools." Two further server-declared capabilities worth knowing by name: **logging** (tracked by the server, sent to the client in structured form) and **completion** (argument autocomplete, offered by the server for its own tool/prompt arguments).

6.2 Attacks at the tool-server level — three categories, each with its own fix

This is the concrete taxonomy underneath the general "connecting to an external tool server extends your trust boundary" principle above — how that extension actually goes wrong:

1. **Malicious server** — e.g., a "document summarizer" tool that quietly exfiltrates every document it touches to an attacker-controlled endpoint. Mitigations: anomaly detection on argument volume (a tool suddenly handling far more data than its stated purpose implies is a signal); a deterministic, idempotent scanning pipeline around anything a tool ingests; least privilege and least sandboxing scope for what the tool can reach; and no token passthrough — never forward a caller's own credential to a downstream server unmodified.
2. **Compromised server** — a previously-trustworthy server ships an update that introduces malicious code. Mitigations: pin the server's version and dependencies rather than always-latest; scan the server and its software bill of materials before trusting an update; least privilege and sandboxing as the standing defense even if a compromise slips through.
3. **Impersonation of a server** — a connection claims to be a trusted server but isn't. Mitigations: verify the connection; bind tokens to the specific server/audience they were minted for; pin and verify server identity rather than trusting whatever answers at a given address.

7. State and persistence

Agent systems carry several distinct kinds of state, each with a different trust and durability profile:

Kind of state	Scope	Trust level	Key risk
Working memory	Single reasoning turn	Ephemeral, in-context	Context-window overflow, truncation
Conversation memory	Single session	Session-scoped	Cross-session leakage if not properly isolated
Checkpoint / proposal state	A specific pending action	Durable, transactional	Must survive retries and partial failure
Episodic memory	Past interactions, summarized	Durable, longer-lived	Can encode a wrong "remembered" fact that then drives future bad actions
Semantic / long-term memory	Facts about an entity across sessions	Durable, cross-session	The most dangerous class — a poisoned write here replays an attack on every future session

Principles:

- **Long-term memory inherits full data-governance obligations** — access control, retention limits, deletion/right-to-erasure support — because it is durable stored data, not a debugging convenience.
- **Memory is a write-policy-gated surface, not an automatic side effect.** What gets promoted from a session into durable memory should be validated and classified before being stored as if it were a trusted fact; otherwise an indirect-injection payload written once into memory replays indefinitely. **Durable truth lives in the system of record, not in the model or its memory.** Anything remembered by the agent should be treated as a hint to be re-verified against live data before it drives a consequential action, not as itself authoritative.
- Three protection rules worth stating as a checklist, not just a principle: user-and-tenant isolation on every memory read/write; every memory must carry a **defined lifetime and expiry** — nothing durable by default; and every stored memory should **cite where it came from**, so a downstream consumer can trace a "remembered" fact back to its source rather than trusting it as free-floating ground truth.

Five authority levels, worth distinguishing whenever reasoning about *whose* authority a given action actually runs under: End User → Service → Delegated → System → Mixed. Most of this section's guidance is really the argument for staying as far left on this spectrum as the operation allows — "End User" is the strongest guarantee (5.3's strongest confused-deputy mitigation), and each step to the right is a deliberate, narrowly-scoped exception, not a default.

Five non-negotiables specifically for multi-agent (supervisor/specialist) architectures — extends section 6's "treat every inter-agent handoff as untrusted" principle into a concrete checklist:

1. **Route on semantics, validate on output** — a supervisor's routing decision is a guess about intent; only the specialist's actual output gets trusted, and only after validation.
2. **Give each specialist the least context it needs** to do its one job — not the full conversation by default.
3. **Least privilege and authority, scoped down per hop** — a specialist's authority should never be broader than what its one job requires, even if the supervisor itself holds more.
4. **Tag every sub-agent's claim back to the supervisor** — attribution has to survive the hand-off, not just exist at the top level.
5. **Every side effect must go through the same idempotency checkpoint**, regardless of which specialist in the graph triggered it — idempotency is a property of the system, not of any one agent's code path.

8. Concurrency and consistency

- **No global transaction spans the model, the tool layer, and downstream systems.** Design explicitly for partial success: a multi-step action can succeed on step one and fail on step two, and there is no built-in rollback.
- **Time-of-check to time-of-use (TOCTOU) is a first-class risk.** State read during planning (e.g., "does this record still look eligible?") can change before the actual write executes. Authorization and business invariants must be re-evaluated **at the point of mutation**, atomically, never trusted from an earlier read.
- **Optimistic concurrency** (a version token captured at read time, checked and updated atomically at write time) is the default pattern for guarding against lost updates when two operators or two concurrent agent turns touch the same entity. For anything more complex than a single record (e.g., a computed "effective permissions" view derived from many contributing records), a **composite version token** — a hash over every contributing record's version — is needed so a write is rejected if *any* input to the original decision has since changed.
- **Pessimistic concurrency is the other tool in the kit, not a worse version of optimistic** — reach for it specifically when the work between read and write includes an external callout that can't be cheaply retried (an outbound API call mid-transaction, for instance): acquire an exclusive lock before touching the record, hold it until commit, block any other writer for that window. Optimistic concurrency assumes conflicts are rare and retry is cheap; pessimistic concurrency is the right choice exactly when retry isn't cheap because a real side effect already happened in between.
- **Retries require idempotency.** Network hops and model calls fail and get retried; without an idempotency key derived from the logical operation, a retry can double-apply an effect (double refund, double grant). The

standard pattern: claim an idempotency key atomically before doing the work; if the key is already claimed, return the previously recorded result instead of re-executing. The key itself needs to be bound to more than just the operation — a key derived only from the logical operation (e.g. `createUser`) is reusable by a *different* caller unless it also incorporates the invoker's own identity, something like `hash(operation + invoker_credentials)`, so the key can't be replayed or reused across principals. For concurrent requests specifically, don't compute-then-check for a duplicate — **insert the key under a unique constraint** and let the constraint itself reject the second concurrent attempt; a check-then-insert has its own race window, an insert with a unique constraint doesn't.

- **Idempotency has a layer problem, not just a mechanism problem.** Request-level idempotency (dedup on a key per call) doesn't automatically give you business-level idempotency — once a step touches multiple downstream systems, dedup on your own table isn't enough on its own, because a crash between "your write committed" and "the other system was told" leaves the two systems disagreeing with no local record of the gap.
- **Serialize writes per logical entity, parallelize everything else.** Reads and writes to independent entities can run concurrently; writes that target the same entity must be ordered (e.g., via a per-entity queue or lock) to preserve concurrency guarantees.
- **Sagas / compensating actions** replace atomic multi-step transactions across systems that don't share a transaction manager: each step should be independently reversible or reconcilable, rather than assuming all steps succeed together.
- **The transactional outbox pattern is the concrete fix for the dual-write problem** — worth being able to draw from memory. One database transaction both makes the business change *and* inserts a row into an outbox table (`BEGIN → create/update record → insert outbox event → COMMIT`, atomically, as one transaction); a separate, independent relay process then reads the outbox and publishes the event to the other system, marking the row delivered only after a confirmed publish. This turns "update my data and tell another system it happened" from two operations that can fail independently into one atomic local commit plus an at-least-once, retryable relay — and because the relay redelivers on any doubt, **the thing it delivers to had better be idempotent to receive**, which is exactly why compensating actions themselves need to be idempotent too: if a compensating "revoke this grant" call times out without you knowing whether it landed, retrying it should be safe by construction, not something to reason about case by case.
- **Verify the post-condition; don't assume the effect.** After a mutation, especially one with multiple possible paths to the same end state (e.g., several ways a permission could be re-granted), re-derive the intended end state and assert it actually holds. A 200-OK response is not proof that the world now looks the way you intended.

Six rules for reasoning about concurrency under pressure, worth having as a single recitable list:

1. Parallelize independent reads.
2. Serialize conflicting writes.
3. Every write is compare-and-swap, not blind overwrite.
4. Never hold a lock, or sit inside a transaction, across a model call or any other slow external callout — that's an availability incident waiting to happen, not a correctness fix.
5. Read your own writes — don't let a caller that just wrote see stale state from a replica or cache on its very next read.
6. Guard invariants explicitly: route consistency-sensitive reads to the primary, not a replica; return the actual post-write state from the call itself rather than re-deriving it; and version-fence reads so a stale read can't silently drive a write.

9. Failure modes

Failures in an agent system are best organized by architectural layer, since the failure signature and the right mitigation differ sharply by layer:

Layer	Representative failure	Typical symptom	Primary mitigation
Channel / experience	Duplicate submission on perceived hang	Duplicate side effects	Idempotency keys per turn
Agent configuration	Overlapping/ambiguous task-scope definitions	Non-deterministic routing	Disjoint scopes, precise routing descriptions, eval gating
Runtime/orchestration	Planning loop (oscillating re-plans)	Latency spikes, cost blowup	Step budgets, loop detection, deterministic fallback
Reasoning (model)	Hallucinated content or a fabricated tool argument	Confident, wrong, well-formed answer with no error	Grounding + citation enforcement; strict schema validation at the gateway
Tool/action execution	Unvalidated argument (e.g., an absurd limit value)	Resource exhaustion or unsafe write	Range/enum validation at the gateway
Data/grounding	Stale retrieval index	Agent cites outdated info with a "passing" grounding check	Freshness monitoring, ingestion-lag alerts
Multi-agent coordination	Cascading retries across specialists	Fan-out amplification	Circuit breakers, bulkheads per specialist
Infrastructure	Model provider degradation	Broad latency/error spike	Fallback routing, graceful degradation

A complementary taxonomy, specific to the intent/topic/action-selection layer — worth having as a fast enumerable list for "what actually goes wrong in practice, not in theory": misclassification; no topic match; multiple competing topic matches; a hallucinated or missing required parameter; a schema or enum violation; an ambiguous entity reference (which record, exactly?); a rate limit hit mid-plan; a grounding check that silently fails open; and prompt injection delivered specifically via grounded content rather than user input. Nine failure shapes, each wanting its own detection signal rather than one generic "error rate" metric — most of them produce a confident, well-formed, wrong answer, not an exception.

A cross-cutting and often-missed category is **slow-burn / silent degradation**: failures that never trip a single threshold alert but compound over days or weeks — gradual quality decay from small prompt/index/model changes stacking, refusal-rate creep, configuration drift between environments, or memory/eval-set staleness. These require monitoring **rates of change against a trailing baseline**, not just absolute thresholds, because each individual signal can look "green" in isolation while the aggregate trend is a real regression.

The most important general failure-analysis habit: **a well-formed, confident, HTTP-200 answer can be entirely wrong.** Conventional error-rate monitoring is blind to this class of failure, which is unique to systems where the decision-maker is a language model rather than deterministic code.

10. Observability

Conventional distributed tracing ("three pillars": traces, logs, metrics) still applies but needs agent-specific enrichment, plus arguably a fourth pillar: **replay artifacts** — a captured record of exactly what the model saw (rendered prompt, retrieved context, tool schemas) for a given decision, because reconstructing "why did it do that" is a recurring operational need that a plain trace doesn't satisfy.

Key differences from a conventional service trace:

- **The trace is session-rooted, not request-rooted** — it spans a whole multi-turn conversation, with turns and reasoning/tool spans nested beneath.
- **The shape of the trace is itself a signal.** Because control flow is synthesized at runtime, a shift in the *distribution* of paths taken (more re-plans, a different tool mix, more refusals) is often the earliest indicator of a regression — frequently before any error-rate metric moves.
- **High-cardinality fields (user IDs, free-text arguments) must never become metric labels** — they belong in traces/logs, sampled and indexed, or you will overwhelm a metrics backend.

A minimal useful signal set: refusal rate (with confidence context), tool-selection entropy (a distributional drift detector — unexpectedly flat or shifted tool usage often means a prompt/catalogue regression), grounding-check failure rate, re-plan/loop count, permission-denied rate (a spike can be either an outage *or* an attack probe), and anomalous argument values.

Alerting discipline: separate *page-worthy* (user-visible harm or security signals — SLO burn rate, guardrail-bypass, permission-denied spikes) from *dashboard-only* (distributional/quality trends). Correlate every alert and every dashboard with a change marker for each of the independently-versioned axes (model version, prompt version, tool-catalogue version, index version, and configuration) — the single most common root cause of a regression is a change on one of these axes, and without a change annotation, diagnosis time balloons.

Traces are sensitive data, not ordinary application logs. They routinely contain user utterances, retrieved record content, and tool arguments — potentially regulated data. Mask at capture time (before writing to the trace store), tier retention (aggregated metrics long-lived; masked traces medium-lived; any unmasked debugging tier short-lived and tightly access-controlled; security audit logs long-lived and immutable), and ensure the trace store can honor a data-deletion request, since it contains personal data like any other store.

11. Scalability considerations

An enterprise agent is a **distributed fan-out topology**, not a single service, and its overall latency is best understood as **multiplicative in the number of reasoning turns, not additive in pipeline stages**. A request that requires the planner to select a tool, observe the result, and select a second tool pays for two full model round-trips plus two guardrail passes — latency budgeting must be done per-turn and summed over the expected turn count for a given workflow.

Key latency components:

- **Inference latency** decomposes into time-to-first-token (dominated by prompt/prefill length) and inter-token latency (dominated by output length). The practical consequence: **the length of the system prompt and grounded context is a tax paid on every single turn** — prompt minimization, output-length capping, and prompt-prefix caching are the primary levers.
- **Retrieval latency** depends on index size and query selectivity; a **semantic cache is dangerous in a multi-tenant, permission-scoped system** unless the requesting principal's authorization context is folded into the cache key — two users can phrase a question near-identically while having very different rights to the answer, and a naive cache keyed only on query text can leak one user's data to another.
- **Action/tool latency** depends on the downstream system being called; a slow synchronous tool call can starve the platform's overall concurrency budget if long-running requests aren't isolated or capped.
- **Backpressure is a security control as much as a performance one.** Without admission control, a burst of agent-driven requests can exhaust downstream concurrency and degrade the whole platform — a self-inflicted denial-of-service condition.

Illustrative latency budget for a representative multi-turn workflow (illustrative numbers only, not measured figures from any real system):

Stage	p50	p95	p99
Ingress + auth	\~20 ms	\~60 ms	\~120 ms
Input guardrail scan	\~40 ms	\~120 ms	\~300 ms
Planning inference (turn 1)	\~700 ms	\~1.8 s	\~3.5 s
Retrieval	\~120 ms	\~400 ms	\~900 ms
Planning inference (turn 2)	\~650 ms	\~1.7 s	\~3.3 s
Authorization check	\~60 ms	\~180 ms	\~400 ms
Action execution	\~150 ms	\~500 ms	\~1.2 s

Stage	p50	p95	p99
Output guardrail scan	\~50 ms	\~150 ms	\~350 ms
Response synthesis (turn 3)	\~500 ms	\~1.3 s	\~2.6 s
End-to-end (excl. human approval wait)	**\~2.3 s**	**\~6.4 s**	**\~13.3 s**

The reading that generalizes: **the reasoning turns dominate both the median and the tail**; collapsing multi-turn plans into fewer round-trips is the single biggest latency lever, traded against harder guardrailing per turn. The p50-to-p99 spread is characteristically wide for LLM-backed pipelines — SLOs should be quoted at p95, and client timeouts set comfortably above p99, not around the average.

Perceived latency and actual latency are different problems with different fixes. If a compile/planning call can't be made faster, the wait can still be made to feel shorter and more trustworthy by streaming real intermediate state as it's produced — a "thinking out loud" pattern — rather than blocking on a monolithic call behind a static "thinking…" indicator. This only works if the underlying process is already designed as granular, inspectable stages (requirement gathering → scaffold → compile, each its own sub-stage) rather than one opaque call; designing for streaming later means designing granular stages now, even before a streaming UI is built.

Parallelizing independent steps within a single plan is a natural next lever once multi-step plans are common. The general pattern: the planner does dependency analysis at compile time (not the executor at run time), grouping steps into stages based on whether a step's inputs depend on another step's output — two steps are parallelizable only if neither consumes the other's result and neither writes to a resource the other reads or writes, the same kind of data-flow dependency graph used in instruction scheduling or a build system's task graph. The runtime, on hitting a parallel stage, fans out all steps concurrently and joins before advancing — the join is a hard synchronization point; the next stage cannot start until every step in the current one completes or definitively fails. **Partial failure inside a parallel stage is the genuinely hard part.** For independent *reads*, the safe default is to let the successful ones complete, surface the one failure explicitly rather than silently dropping it, and show partial results with the gap clearly marked. For independent *writes*, this is substantially harder — it needs either per-step idempotent retry with no cross-step ordering requirement, or a decision about whether the whole stage is all-or-nothing, which then needs a compensating-action/rollback story for steps that already succeeded (the saga pattern, section 8). The honest position: read-only parallel stages are the safe first version of this feature; parallel writes are a materially harder problem worth treating as a separate, later design effort rather than bundling into the same change.

Sharding or partitioning a shared retrieval index is a real trade-off, not a default. A shared index with a tenant filter applied at query time is simplest to build, but query cost scales with the *total* corpus size across all tenants even though any one query only needs one tenant's slice — and it carries the same pre-filter-vs-post-filter leakage risk from 5.9 if the tenant filter isn't applied before the vector search itself. A per-tenant partition (shard) keeps query cost flat as other tenants' data grows, at the cost of partition-management overhead (provisioning, rebalancing uneven tenant sizes, more moving operational parts). This is the same trade-off shape as consistent-hashing-style partitioning in any multi-tenant data system: partitioning trades query-time simplicity for operational complexity, and it's the right trade specifically when per-tenant isolation is already a hard requirement anyway — meaning the "extra" operational cost of partitioning is partially already being paid for the isolation guarantee, so the marginal cost of also getting a scaling benefit from it is lower than building both a filter-based isolation layer *and* a separate scaling fix.

Horizontal scaling of a stateful conversation manager is a genuinely hard question worth reasoning through explicitly rather than assuming a specific implementation: is the component that "owns" a conversation's state pinned to one process (a sticky session), or can any instance pick up any turn? Conversation continuity implies *some* durable state per conversation has to persist between turns. The architecturally clean way to get horizontal scalability without sticky sessions is the durable-checkpoint pattern: state checkpointed to a real store keyed by conversation/thread ID, so any stateless compute instance can pick up the next turn by loading that state fresh, rather than requiring the same process to still be alive. The compute-heavy planning/compile step is comparatively

easy to scale horizontally, since it's closer to stateless-per-invocation given its inputs — the harder scaling problem is state continuity across turns, not raw compute parallelism.

Delivery semantics for any multi-stage ingestion/extraction pipeline are worth reasoning about explicitly in at-least-once vs. exactly-once terms. A reasonable, defensible default position: this kind of pipeline should be **at-least-once with idempotent re-processing** — a document safely re-ingested from any stage produces the same final state, so a mid-pipeline retry or restart is harmless — rather than trying to guarantee exactly-once end-to-end. Exactly-once across a multi-stage asynchronous pipeline is notoriously expensive to actually guarantee, and idempotent re-processing gets the same practical safety without that cost.

12. Design trade-offs

Trade-off	Choose the first when...	Choose the second when...
Retrieval (RAG) vs. direct structured lookup	Answering fuzzy, natural-language questions over unstructured docs	The fact must be exactly correct and current ("what access does this person have right now") — never let retrieval output stand in for an authorization decision
LLM-planned agent vs. deterministic workflow	Open-ended intent capture, flexible front door	Everything after intent is captured — the execution plan should be deterministic, testable code
Standardized tool-protocol server vs. plain internal API	The tool surface is consumed by models across multiple hosts and benefits from standardized discovery/security	The consumer is conventional software with no need for model-facing discovery
Generic tool vs. bespoke narrow action	Low-consequence, high-variety reads or light, tightly-constrained writes	Any high-consequence, irreversible, or regulated operation — never ship an unconstrained generic write tool to an agent with access to anything sensitive
Model-evaluated policy vs. deterministic policy engine	Never, for authorization decisions	Always, for authorization — a model may *suggest*, but permit/deny must come from code
Caller's own credentials vs. a service/integration identity	The caller has a downstream identity — the default	A narrow, unavoidable elevation is required (e.g., a cross-system lookup the caller has no direct path to) — kept as tightly scoped as possible
Single agent vs. multi-agent	The domain is genuinely one cohesive scope	Distinct specialized domains justify the added coordination and trust-surface cost
Long context ("stuff it all in") vs. retrieval-on-demand	Rarely — mostly for small, non-sensitive context	Anything large or sensitive — putting a large amount of data in-context both costs more and expands the injection/exfiltration surface
Synchronous vs. asynchronous execution	Reads and auto-approved low-risk writes	Anything requiring human approval or bulk processing
Optimistic vs. pessimistic concurrency	Conflicts are rare and retry is cheap	The work between read and write includes an external callout that can't be cheaply retried

The unifying principle behind nearly every row: **push flexibility toward the reasoning layer and push determinism toward the execution layer.** The model should have latitude to interpret intent; it should have almost none when it comes to decide-and-act on consequential operations.

13. Generic pseudocode examples

Deterministic authorization middleware, independent of the model's output:

```
def intercept_tool_call(call, caller):
    if not caller.is_authenticated():
        return deny("unauthenticated")

    required_capability = map_tool_to_capability(call.tool_name, call.args)
    if not authz_service.caller_holds(caller, required_capability):
        audit.record(caller, call, "DENY_NO_CALLER_AUTHORITY")
        return deny(f"caller lacks: {required_capability}")

    decision = policy_engine.evaluate(caller, call, required_capability)
    audit.record(caller, call, decision)  # log the decision before any effect
    if decision.effect == "PERMIT":
        return proceed(decision.obligations)
    elif decision.effect == "REQUIRE_APPROVAL":
        return route_to_approval(call, decision)
    else:
        return deny(decision.reason)
```

Idempotent execution wrapper:

```
def execute_once(proposal, do_work):
    first_time = idempotency_store.try_claim(proposal.key, proposal.id)
    if not first_time:
        return idempotency_store.await_result(proposal.key, timeout=30)
    try:
        result = do_work()
        idempotency_store.record_result(proposal.key, result)
        return result
    except Exception as e:
        idempotency_store.record_failure(proposal.key, e)
        raise
```

Optimistic concurrency guard:

```
def mutate_with_version_check(entity_id, expected_version, mutation_fn):
    current = data_store.read(entity_id)
    if current.version != expected_version:
        raise ConcurrencyConflict("state changed since read; re-inspect before mutating")
    return mutation_fn(entity_id)  # conditional on the version check above
```

Permission-aware retrieval (pre-filter, not post-filter):

```
def acl_aware_search(query, caller, k=8):
    query_vector = embed(query)
    visible_scopes = caller.authorization_labels()
    hits = vector_index.search(
        vector=query_vector,
        top_k=k,
        filter={"scope": {"in": visible_scopes}},  # enforced INSIDE the search
    )
```

```
    # belt-and-suspenders: re-check live authorization on the result set
    return [h for h in hits if caller.can_read(h.resource_id)]
```

Tool-selection distribution as a drift detector (observability):

```
import math

def tool_selection_entropy(tool_counts: dict) -> float:
    total = sum(tool_counts.values())
    if total == 0:
        return 0.0
    entropy = 0.0
    for count in tool_counts.values():
        if count == 0:
            continue
        p = count / total
        entropy -= p * math.log2(p)
    return entropy  # compare against a trailing baseline; alert on large deviation
```

Deterministic policy decision (illustrative; production systems typically use a dedicated policy-as-data engine):

```
def evaluate(request) -> Decision:
    if request.action == "grant" and request.target_capability in SENSITIVE_SET:
        if not request.caller_holds_admin_capability:
            return Decision("DENY", "sensitive grant requires elevated authority")
        return Decision("REQUIRE_APPROVAL", "sensitive grant")
    if request.batch_size > MAX_BATCH:
        return Decision("DENY", "exceeds blast-radius limit")
    if request.action == "inspect":
        return Decision("PERMIT")  # reads are already access-constrained upstream
    return Decision("REQUIRE_APPROVAL", "default-deny-to-human for writes")
```

14. Principal-level engineering lessons

1. **The model proposes; deterministic systems authorize.** Keeping the permit/deny decision outside the model is what makes the system certifiable at all.
2. **Retrieved content is data, never authority.** Anything fetched from an index or document is untrusted input to reason *about*, never an instruction to obey.
3. **Authentication and authorization are different questions.** Conflating "they proved who they are" with "they may do this" is how over-broad access leaks in.
4. **Authorization belongs at execution time, against live state — not at plan time.** A plan-time check is advisory; only an execution-time check is enforcement.
5. **Never trust the planner as a security boundary.** Anything influenceable by attacker-controlled text cannot be the thing standing between a request and a harmful effect.
6. **Tool descriptions and tool outputs are both untrusted input.** One shapes what the model decides to call; the other re-enters its context. Both are viable injection vectors and should be treated accordingly.
7. **Grounding improves factuality, not authorization.** A retrieved fact is never a substitute for a permission check.
8. **Every write must be idempotent or safely retryable.** Networks time out and models re-emit; the same intent must apply at most once.

9. **Human approval complements authorization; it never replaces it.** A human's click does not make an unauthorized action authorized.
10. **Use the caller's own authority wherever possible.** It bounds the blast radius by construction and makes downstream authorization apply "for free."
11. **Autonomy should decrease as consequence increases.** Reads can be fully automatic; irreversible or high-value actions demand a human and a throttle.
12. **Never grant a capability you cannot independently constrain.** If you can't put a deterministic limit around a tool, you can't safely hand it to a non-deterministic caller.
13. **Deny by default; permit by exception**, for every action and every actor.
14. **Minimize what enters the model's context window.** Every token of sensitive data in context is both a cost and an exfiltration/injection surface.
15. **Prefer narrow, verb-specific tools over generic operations.** Specificity is what makes policy enforcement tractable and blast radius provable.
16. **Log the decision, not just the outcome.** "It succeeded" is not an audit trail; "it was permitted because X" is.
17. **Bound the blast radius of every action** — cap batch sizes, rate-limit sensitive operations, cool down after bursts.
18. **Attribute every action to a real, accountable identity.** Un-attributable power is unauditable power.
19. **Guard shared state with explicit concurrency control**, never with "the model will notice and avoid the conflict."
20. **Fail closed, degrade gracefully.** Availability pressure must never be resolved by relaxing a security decision.
21. **Separate the reasoning plane from the control plane as independently deployable units**, not just as a logical distinction inside one process.
22. **Design for the adversarial case first, then relax — never the reverse.** Systems built for the happy path acquire their security posture by accident and lose it the same way.

15. Questions I should be able to answer in a system-design discussion

Foundational

- What is the difference between the model's role and the runtime's role when a "tool call" happens?
- Why is a well-formed, confident, error-free answer not sufficient evidence that an agent behaved correctly?
- What's the difference between a soft guardrail and a hard guardrail, and which one carries security guarantees?

Senior

- A retrieval index has gone stale. What is the concrete failure mode, and what's the correct mitigation?
- Why must side-effecting tools be idempotent, and how do you implement that in practice?
- Walk through what "the model decides to call a tool" actually triggers, step by step, up to the point of a real side effect.
- What's the difference between direct and indirect prompt injection, with a concrete example of each?

Staff

- Intent classification is probabilistic and will sometimes misroute. What's your containment strategy, independent of routing accuracy?
- "We validate permissions in the tool." Where exactly, is it deterministic, what happens if the check errors, and how does it behave under concurrent access?
- Walk through the correct handling when a tool call to an external system times out but may have actually succeeded.
- Name the trust boundaries in a typical deployment and what each one requires for authentication/authorization.

- A diagnostic plan naturally decomposes into one setup step and several independent data-gathering steps. Design the executor-side mechanism that safely parallelizes the independent steps, including the partial-failure case.

Principal

- Design the trust and consistency model for adding long-term, cross-session memory. What breaks at scale?
- Two concurrent operations touch the same entity. What breaks, and how do you make the system correct without relying on the model to coordinate?
- Design the observability needed to reconstruct a single production turn end-to-end, and reconcile that with data-minimization/privacy constraints.
- Argue against "just give the agent broad access so it can help with anything," from a blast-radius and confused-deputy standpoint.
- Compare bespoke, narrow actions against generic, broadly-parameterized tools — when is each appropriate, and what changes about the security analysis?
- Design the tenant-scoped retrieval system for a shared knowledge index at very large multi-tenant scale — state the shared-index-with-filter vs. per-tenant-shard trade-off explicitly.

Security-architect tier

- You're connecting the agent to an external tool server over a standardized protocol. Interrogate the trust model: server trust, description trust, token scope, output handling, and change control.
- Walk through a complete confused-deputy attack end-to-end and name every independent control that should stop it, so that no single failure is catastrophic.
- What exactly does data masking protect against, what are its limits, and how does it interact with an active exfiltration attempt?
- Define "fail closed" precisely across authorization, guardrails, and dependency failures, and give a concrete example where "fail open" would be catastrophic.
- You're signing off on an autonomous, revenue-affecting agent. Enumerate the non-negotiable controls and the residual risks you would still document even after they're all implemented.
- Name the eight vectors in the grounding-specific threat model and give a one-line mitigation for each without notes.

16. One-page revision sheet

The one-sentence thesis: an LLM-driven agent is a new, especially potent confused-deputy surface, and the entire architecture exists to make sure that surface can propose but never unilaterally authorize.

Core boundary: reasoning plane (untrusted, non-deterministic) vs. control plane (trusted, deterministic). Everything crossing it is data, not authority.

The four authority boundaries where things actually go wrong: User→Agent, Planner→Router, Router→Tool, Tool→Data Store. The Router→Tool hop is where end-user identity becomes system authority, and the Tool→Data Store hop is the one most often wrongly assumed to be already covered by an upstream check.

The five things that must always be deterministic, never left to the model:

1. Authorization (permit/deny)
2. Argument/schema validation
3. Idempotency and concurrency control
4. Blast-radius limits (batch caps, rate limits)
5. Audit logging of the decision, not just the outcome

The three untrusted-input categories, all requiring the same treatment (segregate, never treat as instruction):

- User utterances (direct injection)
- Retrieved/grounded content (indirect injection, plus the eight-vector grounding threat model in 5.9)
- Tool descriptions and tool outputs (supply-chain / output injection)

The tiering heuristic: autonomy decreases as consequence increases. Read → automatic. Reversible low-impact write → automatic + logged. Consequential write → confirm. High-value/irreversible → human approval by a distinct approver + rate limiting.

The concurrency heuristic: re-check authorization and business invariants at the point of mutation, atomically, using a version token — never trust a decision made at plan time against now-stale state. For the dual-write problem specifically, reach for a transactional outbox rather than hoping two separate writes stay in sync.

The observability heuristic: correlate everything by one trace/turn ID; monitor *distributions* (tool-selection entropy, refusal rate, re-plan count) in addition to error rate, because agents fail semantically, not just with exceptions; mask sensitive data at capture time, not at read time.

The tool-design heuristic: narrow, typed, verb-specific tools are easy to authorize and audit; generic tools are a general-purpose exploit unless constrained by an explicit, deterministic capability envelope (allow-listed objects/fields, hard value constraints, platform authorization as backstop). A tool-calling protocol connection is only as trustworthy as the weakest of: server trust, description trust, token scope, and output handling.

The scaling heuristic: reasoning turns dominate latency, not pipeline stages; collapse round-trips before anything else. Parallelize independent plan steps only after solving the partial-failure case for writes. Choose a shared index with a tenant filter vs. a per-tenant shard as an explicit trade-off, not a default.

The five independently-versioned change axes (any one of them can silently regress quality or security): model version, prompt/instruction version, tool/action definition version, grounding/retrieval configuration, and underlying data schema. Every change should be versioned, evaluated against a security-relevant regression suite, and independently rollback-able (dark launch → shadow → canary → percentage → full, with an instant kill switch armed throughout).

The one question that catches almost every design flaw: "*If the model is fully compromised by an attacker right now, what is the worst thing it can actually cause to happen?*" If the answer is bounded, small, attributable, and reversible, the architecture is sound. If it isn't, some layer is trusting the model as a security boundary.
